

Ejercicio 1: Decisión de Evaluación de Expresiones mediante Programación Dinámica

Este ejercicio es un clásico de programación dinámica sobre intervalos. La clave no es la multiplicación rara. La clave es: *"como los paréntesis cambian el resultado, hay que probar todas las particiones posibles"*. Y eso grita: DP sobre subcadenas.

1. Qué hay que decidir

Dada una expresión:

$$x = x_1 x_2 \dots x_n$$

Queremos saber si existe alguna parentización cuyo resultado final de evaluar la expresión sea el carácter a .

2. Idea clave

Como la operación dada en la tabla no es asociativa:

$$(a \cdot b) \cdot c \neq a \cdot (b \cdot c)$$

Debemos considerar obligatoriamente todas las formas posibles de dividir la expresión para evaluar los subproblemas.

3. Estado de la Programación Dinámica

Definimos $DP[i][j]$ como:

$$DP[i][j] = \{\text{conjunto de valores posibles que puede producir la subcadena } x_i \dots x_j\}$$

Ejemplo:

$DP[2][4] = \{a, c\}$ significa que, usando distintos paréntesis, la subcadena $x_2 x_3 x_4$ puede evaluarse como a o como c .

4. Casos base

Una subexpresión de una sola letra se evalúa únicamente como ella misma:

$$DP[i][i] = \{x_i\} \quad \forall i \in \{1, 2, \dots, n\}$$

5. Ecuación de Recurrencia

Para calcular el estado de una subcadena $DP[i][j]$ (con $i < j$), probamos todos los puntos de corte posibles k dentro del intervalo:

$$k \in [i, j - 1]$$

Esto equivale a partir la expresión en dos bloques: $(x_i \dots x_k) \cdot (x_{k+1} \dots x_j)$.

Tomamos todos los valores posibles que puede devolver el lado izquierdo, todos los del lado derecho, y les aplicamos la tabla de multiplicar:

$$DP[i][j] = \bigcup_{k=i}^{j-1} \{u \cdot v \mid u \in DP[i][k], v \in DP[k+1][j]\}$$

6. Algoritmo

A continuación se presenta el pseudocódigo que decide de manera eficiente la viabilidad de la expresión:

```

Funcion EsPosibleEvaluarA(x, n)
Inicio
    // Matriz DP de conjuntos vacíos
    Crear matriz DP de tamaño (n + 1) x (n + 1)

    // 1. Inicializar casos base (longitud 1)
    Para i <- 1 Hasta n Con Paso 1 Hacer
        DP[i][i] <- { x[i] }
    FinPara

    // 2. Rellenar por longitud de subintervalo (L varía de 2 a n)
    Para L <- 2 Hasta n Con Paso 1 Hacer
        Para i <- 1 Hasta n - L + 1 Con Paso 1 Hacer
            j <- i + L - 1
            DP[i][j] <- {} // Inicializar como conjunto vacío

            // Probar todos los puntos de corte k
            Para k <- i Hasta j - 1 Con Paso 1 Hacer
                // Producto cartesiano de los valores alcanzables
                Para cada u en DP[i][k] Hacer
                    Para cada v en DP[k+1][j] Hacer
                        resultado <- Multiplicar(u, v)
                        DP[i][j] <- DP[i][j] U { resultado }
                    FinPara
                FinPara
            FinPara
        FinPara

    // 3. Respuesta final: ¿el carácter 'a' es alcanzable en la cadena completa?
    Retornar ('a' pertenece a DP[1][n])
FinFuncion

Funcion Multiplicar(izq, der)
Inicio

```

```

Si izq = 'a' Entonces
  Si der = 'a' Entonces Retornar 'b'
  Si der = 'b' Entonces Retornar 'b'
  Si der = 'c' Entonces Retornar 'a'
Sino Si izq = 'b' Entonces
  Si der = 'a' Entonces Retornar 'c'
  Si der = 'b' Entonces Retornar 'b'
  Si der = 'c' Entonces Retornar 'a'
Sino Si izq = 'c' Entonces
  Si der = 'a' Entonces Retornar 'a'
  Si der = 'b' Entonces Retornar 'c'
  Si der = 'c' Entonces Retornar 'c'
FinSi
FinFuncion

```

7. Respuesta final

Existe al menos una parentización válida que da como resultado el carácter a si y solo si:

$$a \in DP[1][n]$$

8. Complejidad

Aquí reside la importancia teórica de esta resolución de Programación Dinámica:

- **Número de estados (subcadenas):** La tabla DP tiene un tamaño de $n \times n$ intervalos, lo que requiere calcular exactamente:

$$O(n^2) \text{ estados}$$

- **Coste por estado:** Para resolver cada celda $DP[i][j]$, probamos todos los cortes posibles k ($O(n)$ opciones). Para cada corte, cruzamos a lo sumo los elementos de la izquierda con los de la derecha. Como el alfabeto tiene un tamaño constante $|\Sigma| = 3$, el producto cartesiano realiza como máximo $3 \times 3 = 9$ operaciones de coste constante. Por tanto, resolver cada celda consume un tiempo de:

$$O(n)$$

9. Complejidad Total

Multiplicando el coste de calcular todos los estados por el tiempo requerido para resolver cada uno de ellos, obtenemos una complejidad asintótica de:

$$T(n) \in \Theta(n^3)$$